

```
#####
#
#   COURS : BASES DU RAISONNEMENT MATHÉMATIQUE
#   Théorie · Démonstrations · Code Python natif
#
#####
#
# COMMENT LIRE CE COURS ?
#
# Comme un cours de programmation sur les variables et les fonctions :
# on comprend POURQUOI ça existe, puis COMMENT ça marche,
# puis on le voit EN VRAI dans du code Python.
#
# Chaque notion suit ce plan :
# [THÉORIE]      – L'idée, pourquoi ça existe
# [DÉFINITION]   – La règle formelle
# [DÉMONSTRATION] – Preuve pas à pas
# [EXEMPLE]      – Exemple concret
# [CODE PYTHON]  – Programme qui illustre la notion
# [PIÈGE]        – Erreur classique à éviter
#
#####
```

## PARTIE 1 – CE QU'EST UN RAISONNEMENT MATHÉMATIQUE

### 1. LES PROPOSITIONS – la brique de base

#### [THÉORIE]

En programmation, la brique de base c'est la variable : elle contient une valeur. En mathématiques, la brique de base c'est la PROPOSITION : c'est une phrase qui est soit VRAIE, soit FAUSSE. Pas les deux. Pas "ça dépend". Exactement comme un booléen en code (True / False).

Exemples de propositions :

|                        |                                                                                   |
|------------------------|-----------------------------------------------------------------------------------|
| "2 + 2 = 4"            | → VRAIE                                                                           |
| "7 est un nombre pair" | → FAUSSE                                                                          |
| "x > 0"                | → dépend de x (c'est un PRÉDICAT, pas encore une proposition pure – on y revient) |

Ce qui N'EST PAS une proposition :

|                         |                                         |
|-------------------------|-----------------------------------------|
| "Les maths sont belles" | → opinion, pas de valeur de vérité fixe |
| "x + 1"                 | → expression, pas une phrase            |

#### [DÉFINITION]

Une proposition P est un énoncé auquel on peut attribuer exactement une valeur de vérité : VRAI (V) ou FAUX (F).

#### [CODE PYTHON]

# Les propositions = les booléens en Python

```
p = (2 + 2 == 4)      # True  – proposition vraie
q = (7 % 2 == 0)      # False – 7 n'est pas pair
r = (10 > 3)          # True
```

```
print("p:", p)        # True
print("q:", q)        # False
print("r:", r)        # True
```

#### [PIÈGE]

"x > 0" n'est pas ENCORE une proposition car x n'a pas de valeur. C'est une proposition OUVERTE (prédicat). Elle devient une proposition dès qu'on donne une valeur à x : si x = 5, alors "5 > 0" est VRAIE.

### 2. LES CONNECTEURS LOGIQUES – assembler des propositions

## [THÉORIE]

Comme en programmation où on combine des conditions avec "and", "or", "not", on combine des propositions avec des CONNECTEURS LOGIQUES. Le résultat est lui aussi une proposition (vraie ou fausse).

Les 4 connecteurs fondamentaux :

$\neg P$  : NON P (négation)  
 $P \wedge Q$  : P ET Q (conjonction)  
 $P \vee Q$  : P OU Q (disjonction – OU inclusif)  
 $P \Rightarrow Q$  : SI P ALORS Q (implication)  
 $P \Leftrightarrow Q$  : P si et seulement si Q (équivalence)

## [DÉFINITION – Tables de vérité]

NÉGATION  $\neg P$  :

$P = V \rightarrow \neg P = F$   
 $P = F \rightarrow \neg P = V$

ET  $P \wedge Q$  :

$V \wedge V = V$  (vrai seulement si LES DEUX sont vrais)  
 $V \wedge F = F$   
 $F \wedge V = F$   
 $F \wedge F = F$

OU  $P \vee Q$  :

$V \vee V = V$  (vrai si AU MOINS UN est vrai)  
 $V \vee F = V$   
 $F \vee V = V$   
 $F \vee F = F$

IMPLICATION  $P \Rightarrow Q$  :

$V \Rightarrow V = V$   
 $V \Rightarrow F = F$  ← le seul cas FAUX (promesse tenue → résultat vrai)  
 $F \Rightarrow V = V$  ← hypothèse fausse, on peut conclure n'importe quoi  
 $F \Rightarrow F = V$

ÉQUIVALENCE  $P \Leftrightarrow Q$  :

$V \Leftrightarrow V = V$  (même valeur de vérité des deux côtés)  
 $V \Leftrightarrow F = F$   
 $F \Leftrightarrow V = F$   
 $F \Leftrightarrow F = V$

## [EXEMPLE]

$P$  = "il pleut"       $Q$  = "je prends mon parapluie"

$P \Rightarrow Q$  : "S'il pleut, je prends mon parapluie"

Ce n'est FAUX que si il pleut ET que je n'ai pas pris mon parapluie.

Si il ne pleut pas, la promesse n'est ni tenue ni rompue → VRAI.

## [CODE PYTHON]

---

# Connecteurs logiques en Python

$P = \text{True}$   
 $Q = \text{False}$

```
non_P = not P          # Négation
P_et_Q = P and Q       # Conjonction
P_ou_Q = P or Q        # Disjonction
implique = (not P) or Q # Implication  $P \Rightarrow Q$  (pas d'opérateur natif)
equiv = P == Q         # Équivalence
```

```
print("¬P      :", non_P)    # False
print("P ∧ Q   :", P_et_Q)   # False
print("P ∨ Q   :", P_ou_Q)   # True
print("P ⇒ Q   :", implique) # False (P vrai, Q faux)
print("P ⇔ Q   :", equiv)    # False
```

# Afficher la table de vérité complète pour  $P \wedge Q$

```
print("\nTable de vérité de  $P \wedge Q$  :")
print(f"{'P':<6} {'Q':<6} {'P∧Q'}")
for p in [True, False]:
    for q in [True, False]:
        print(f"{str(p):<6} {str(q):<6} {str(p and q)}")
```

---

#### [PIÈGE]

L'implication  $P \rightarrow Q$  n'est PAS "P cause Q".  
Elle dit juste : "il est impossible que P soit vrai et Q faux en même temps".  
Si P est faux, l'implication est toujours vraie, même si Q est absurde.  
Exemple : "Si  $1 = 2$ , alors la Lune est en fromage" est mathématiquement VRAI.

---

### 3. LES QUANTIFICATEURS – parler de tous ou d'un seul

---

#### [THÉORIE]

En programmation on dit souvent "pour chaque élément de la liste, faire..."  
C'est exactement le quantificateur universel  $\forall$  (pour tout).  
Et "il existe au moins un élément tel que..." c'est  $\exists$  (il existe).  
Ces deux symboles transforment un prédicat ouvert en proposition.

#### [DÉFINITION]

$\forall x \in E, P(x) \rightarrow$  "Pour tout  $x$  dans  $E$ ,  $P(x)$  est vraie"  
Vraie si  $P(x)$  est vraie pour CHAQUE élément de  $E$ .  
 $\exists x \in E, P(x) \rightarrow$  "Il existe au moins un  $x$  dans  $E$  tel que  $P(x)$  est vraie"  
Vraie dès qu'ON TROUVE UN seul  $x$  qui vérifie  $P$ .  
 $\exists! x \in E, P(x) \rightarrow$  "Il existe un unique  $x$  dans  $E$  tel que  $P(x)$ "

#### [EXEMPLE]

$\forall x \in \mathbb{R}, x^2 \geq 0 \rightarrow$  VRAIE (le carré est toujours positif)  
 $\exists x \in \mathbb{R}, x^2 = 2 \rightarrow$  VRAIE ( $x = \sqrt{2}$  fonctionne)  
 $\forall x \in \mathbb{R}, x^2 = 2 \rightarrow$  FAUSSE ( $x = 1$  ne marche pas)  
 $\exists! x \in \mathbb{R}, x + 5 = 0 \rightarrow$  VRAIE (uniquement  $x = -5$ )

#### [DÉMONSTRATION] de $\forall x \in \mathbb{R}, x^2 \geq 0$

Soit  $x$  un réel quelconque.  
Cas 1 :  $x \geq 0 \rightarrow x^2 = x \cdot x \geq 0$  (produit de deux positifs)  
Cas 2 :  $x < 0 \rightarrow x^2 = (-|x|)^2 = |x|^2 \geq 0$  (produit de deux négatifs = positif)  
Cas 3 :  $x = 0 \rightarrow x^2 = 0 \geq 0$   
Dans tous les cas  $x^2 \geq 0$ .  $\square$

#### [NÉGATION des quantificateurs – très important]

$\neg(\forall x, P(x)) = \exists x, \neg P(x)$  ("pas pour tous" = "il en existe un qui échoue")  
 $\neg(\exists x, P(x)) = \forall x, \neg P(x)$  ("il n'en existe aucun" = "tous échouent")

C'est comme en code :

"il n'est pas vrai que tous les éléments  $> 0$ "  
= "il existe au moins un élément  $\leq 0$ "

#### [CODE PYTHON]

---

```
# Simuler  $\forall$  et  $\exists$  sur une liste finie

nombres = list(range(-5, 6)) # [-5, -4, ..., 4, 5]

#  $\forall x \in \text{nombres}, x^2 \geq 0$  ?
pour_tout = all(x**2 >= 0 for x in nombres)
print("∀x, x²≥0 :", pour_tout) # True

#  $\exists x \in \text{nombres}, x^2 = 9$  ?
il_existe = any(x**2 == 9 for x in nombres)
print("∃x, x²=9 :", il_existe) # True (x=3 et x=-3)

# Trouver les témoins (les x qui vérifient la propriété)
temoins = [x for x in nombres if x**2 == 9]
print("Témoins pour x²=9 :", temoins) # [3, -3]

# Négation :  $\neg(\forall x, x > 0) = \exists x, x \leq 0 \rightarrow$  trouver le contre-exemple
contre_exemple = next((x for x in nombres if not (x > 0)), None)
print("Contre-exemple pour ∀x, x>0 :", contre_exemple) # -5 (premier trouvé)
```

---

#### [PIÈGE]

Pour PROUVER  $\forall x, P(x)$  il faut le montrer pour UN  $x$  QUELCONQUE (général).  
Pour RÉFUTER  $\forall x, P(x)$  il suffit d'UN seul contre-exemple.  
Pour PROUVER  $\exists x, P(x)$  il suffit d'EXHIBER un exemple qui marche.

Pour RÉFUTER  $\exists x, P(x)$  il faut montrer que AUCUN  $x$  ne fonctionne.

---

#### 4. LES TYPES DE DÉMONSTRATIONS – les outils du mathématicien

---

##### [THÉORIE]

En programmation, pour résoudre un problème tu as plusieurs stratégies :  
boucle for, récursion, divide and conquer...  
En maths, pour PROUVER une propriété tu as aussi des stratégies  
bien définies. Connaître ces stratégies, c'est savoir "penser en maths".

---

##### 4.1 PREUVE DIRECTE – "aller droit au but"

---

##### [THÉORIE]

On part de l'hypothèse  $P$  et on arrive à  $Q$  par une chaîne logique.  
C'est comme exécuter un algorithme ligne par ligne.

Structure : "Supposons  $P$ . Alors ... Donc  $Q$ ."

##### [EXEMPLE] Théorème : La somme de deux entiers pairs est paire.

Soient  $a$  et  $b$  deux entiers pairs.  
Par définition du pair :  $a = 2m$  et  $b = 2n$  pour des entiers  $m, n$ .  
Alors :  $a + b = 2m + 2n = 2(m + n)$   
Donc  $a + b$  est de la forme  $2 \times (\text{entier})$ , donc  $a + b$  est pair.  $\square$

##### [CODE PYTHON]

---

```
# Vérification expérimentale de la preuve directe
def est_pair(n):
    return n % 2 == 0

# Tester : somme de deux pairs = pair
resultats = []
for a in range(0, 20, 2):      # pairs de 0 à 18
    for b in range(0, 20, 2):  # pairs de 0 à 18
        assert est_pair(a + b), f"ERREUR : {a}+{b} n'est pas pair !"
        resultats.append((a, b, a+b))

print("Tous les tests passent : somme de deux pairs = pair ✓")
print("Exemples :", resultats[:5])
```

---

---

##### 4.2 PREUVE PAR CONTRAPOSÉE – "retourner l'implication"

---

##### [THÉORIE]

L'implication  $P \Rightarrow Q$  est LOGIQUEMENT ÉQUIVALENTE à  $\neg Q \Rightarrow \neg P$ .  
C'est la CONTRAPOSÉE.  
Parfois il est plus facile de prouver  $\neg Q \Rightarrow \neg P$  que  $P \Rightarrow Q$  directement.

Analogie code :  
if  $P$ : do  $Q$   
est équivalent à :  
if not  $Q$ : il n'aurait pas dû y avoir  $P$

##### [DÉMONSTRATION] de l'équivalence $P \Rightarrow Q \Leftrightarrow \neg Q \Rightarrow \neg P$

Supposons  $P \Rightarrow Q$  vraie.  
Soit  $\neg Q$  vraie ( $Q$  est faux).  
Si  $P$  était vrai, alors  $Q$  serait vrai (par hypothèse), contradiction avec  $\neg Q$ .  
Donc  $P$  est faux, c'est-à-dire  $\neg P$  est vrai. Donc  $\neg Q \Rightarrow \neg P$ .  $\square$

##### [EXEMPLE] Théorème : Si $n^2$ est pair, alors $n$ est pair.

Contraposée : Si  $n$  est impair, alors  $n^2$  est impair.  
Preuve de la contraposée :  
 $n$  impair  $\rightarrow n = 2k + 1$  pour un entier  $k$   
 $n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$   
Donc  $n^2$  est de la forme  $2 \times (\text{entier}) + 1$  : impair.  $\square$

##### [CODE PYTHON]

---

```
# Vérifier : si  $n^2$  est pair  $\leftrightarrow$   $n$  est pair
def verifier_contraposée(limite):
    for n in range(-limite, limite + 1):
        n_carre_pair = (n**2 % 2 == 0)
        n_pair = (n % 2 == 0)
        #  $n^2$  pair  $\square$   $n$  pair
        if n_carre_pair != n_pair:
            print(f"CONTRE-EXEMPLE : n={n}")
            return False
    return True

ok = verifier_contraposée(100)
print("n2 pair  $\square$  n pair vérifiée sur [-100, 100] :", ok)
```

---

#### 4.3 PREUVE PAR L'ABSURDE (CONTRADICTION) – "supposer le contraire et trouver l'erreur"

---

##### [THÉORIE]

Pour prouver  $P$ , on suppose  $\neg P$  (le contraire), et on montre que ça mène à une contradiction (une chose à la fois vraie et fausse).  
Comme en debug : si le programme plante avec cette hypothèse, l'hypothèse est forcément fausse.

Structure :

- "Supposons par l'absurde que  $\neg P$ .
- Alors ... [chaîne de raisonnements] ...
- Donc  $A$  et  $\neg A$  en même temps : CONTRADICTION.
- Donc  $P$  est vrai."  $\square$

##### [EXEMPLE] Théorème : $\sqrt{2}$ est irrationnel.

Supposons par l'absurde que  $\sqrt{2} \in \mathbb{Q}$ .  
Alors  $\sqrt{2} = p/q$  avec  $p, q \in \mathbb{Z}$ ,  $q \neq 0$ ,  $\text{pgcd}(p, q) = 1$  (fraction irréductible).

$\sqrt{2} = p/q$   
 $\rightarrow 2 = p^2/q^2$  (on élève au carré)  
 $\rightarrow p^2 = 2q^2$  (on multiplie par  $q^2$ )  
 $\rightarrow p^2$  est pair (divisible par 2)  
 $\rightarrow p$  est pair (si  $p^2$  est pair,  $p$  est pair – preuve contraposée ci-dessus)  
 $\rightarrow p = 2k$  pour un entier  $k$

Substituons  $p = 2k$  :  
 $\rightarrow (2k)^2 = 2q^2$   
 $\rightarrow 4k^2 = 2q^2$   
 $\rightarrow q^2 = 2k^2$   
 $\rightarrow q^2$  est pair  $\rightarrow q$  est pair

Donc  $p$  ET  $q$  sont tous deux pairs.  
Mais  $\text{pgcd}(p, q) = 1$  impose qu'ils n'ont pas de facteur commun.  
CONTRADICTION.  
Donc  $\sqrt{2} \notin \mathbb{Q}$ .  $\square$

##### [CODE PYTHON]

---

```
import math
from fractions import Fraction

# Illustration : approcher  $\sqrt{2}$  par des rationnels de plus en plus précis
# montre qu'on ne l'atteint jamais exactement

print("Approximations de  $\sqrt{2}$  par fractions :")
valeur_exacte = math.sqrt(2)

# Fractions convergentes de  $\sqrt{2}$  (développement en fraction continue)
convergentes = [(1,1),(3,2),(7,5),(17,12),(41,29),(99,70),(239,169)]

for p, q in convergentes:
    approx = p / q
    erreur = abs(approx - valeur_exacte)
    print(f" {p}/{q} = {approx:.10f} | erreur = {erreur:.2e}")

print(f"\n $\sqrt{2}$  exact = {valeur_exacte:.15f}")
print("On s'approche mais on n'arrive JAMAIS à la valeur exacte en rationnel.")
```

---

---

#### 4.4 PREUVE PAR RÉCURRENCE – "dominos mathématiques"

---

##### [THÉORIE]

C'est l'équivalent mathématique de la RÉCURSION en programmation.

Pour prouver qu'une propriété  $P(n)$  est vraie pour tout entier  $n \geq n_0$  :

ÉTAPE 1 – Initialisation : montrer que  $P(n_0)$  est vraie.

(pousser le premier domino)

ÉTAPE 2 – Hérédité : montrer que SI  $P(k)$  est vraie

ALORS  $P(k+1)$  est vraie.

(chaque domino renverse le suivant)

Si ces deux étapes sont prouvées, alors  $P(n)$  est vraie pour tout  $n \geq n_0$ .

[EXEMPLE] Théorème :  $\forall n \in \mathbb{N}^*, 1 + 2 + 3 + \dots + n = n(n+1)/2$

Notons  $P(n)$  : " $1 + 2 + \dots + n = n(n+1)/2$ "

INITIALISATION –  $P(1)$  :

Membre gauche : 1

Membre droit :  $1 \times 2 / 2 = 1$

Égalité vérifiée.  $P(1)$  est vraie. ✓

HÉRÉDITÉ – Supposons  $P(k)$  vraie pour un  $k \geq 1$  :

$1 + 2 + \dots + k = k(k+1)/2$  ← hypothèse de récurrence

Montrons  $P(k+1)$  :  $1 + 2 + \dots + k + (k+1) = (k+1)(k+2)/2$

Membre gauche :

$[1 + 2 + \dots + k] + (k+1)$

$= k(k+1)/2 + (k+1)$

← on utilise l'hypothèse  $P(k)$

$= (k+1)[k/2 + 1]$

← on factorise  $(k+1)$

$= (k+1)[(k+2)/2]$

$= (k+1)(k+2)/2$

C'est bien le membre droit de  $P(k+1)$ . ✓

Par récurrence,  $P(n)$  est vraie pour tout  $n \geq 1$ . □

##### [CODE PYTHON]

---

# Illustration de la récurrence : somme  $1+2+\dots+n$

```
def somme_naive(n):  
    """Calcul direct (boucle)"""  
    return sum(range(1, n + 1))
```

```
def somme_formule(n):  
    """Formule de Gauss :  $n(n+1)/2$ """  
    return n * (n + 1) // 2
```

# Vérification : les deux méthodes donnent le même résultat

```
print("Vérification de la formule de Gauss :")
```

```
for n in range(1, 20):  
    assert somme_naive(n) == somme_formule(n), f"Erreur à n={n}"  
    print(f" n={n:2d} | somme={somme_naive(n):4d} | formule={somme_formule(n):4d} ✓")
```

# La récursion Python MIME exactement la récurrence mathématique

```
def somme_recursive(n):  
    """Copie exacte de la preuve par récurrence"""  
    if n == 1: # Cas de base (initialisation)  
        return 1  
    return somme_recursive(n - 1) + n # Hérédité :  $S(n) = S(n-1) + n$ 
```

```
print("\nSomme récursive de 1 à 10 :", somme_recursive(10)) # 55
```

---

##### [PIÈGE]

L'hérédité SEULE ne suffit pas : il faut AUSSI l'initialisation.

Et l'initialisation SEULE ne suffit pas : il faut AUSSI l'hérédité.

Les deux sont indispensables, exactement comme une boucle a besoin d'une condition de départ ET d'une condition d'arrêt.

---

## 5. LES ENSEMBLES – regrouper des objets

---

### [THÉORIE]

Un ensemble c'est comme une liste Python, mais sans doublons et sans ordre.  
C'est la structure de base en mathématiques : tout objet mathématique vit dans un ensemble (les entiers, les fonctions, les matrices...).

### [DÉFINITION]

Un ensemble  $E$  est une collection d'objets appelés éléments.

$x \in E$  :  $x$  appartient à  $E$

$x \notin E$  :  $x$  n'appartient pas à  $E$

$A \subseteq B$  :  $A$  est un sous-ensemble de  $B$  (tout élément de  $A$  est dans  $B$ )

$A = B$  :  $A$  et  $B$  ont exactement les mêmes éléments

Opérations :

$A \cup B = \{x \mid x \in A \text{ OU } x \in B\}$  union

$A \cap B = \{x \mid x \in A \text{ ET } x \in B\}$  intersection

$A \setminus B = \{x \mid x \in A \text{ ET } x \notin B\}$  différence

$A^c = \{x \in \Omega \mid x \notin A\}$  complémentaire dans  $\Omega$

### [EXEMPLE]

$A = \{1, 2, 3, 4\}$      $B = \{3, 4, 5, 6\}$

$A \cup B = \{1, 2, 3, 4, 5, 6\}$

$A \cap B = \{3, 4\}$

$A \setminus B = \{1, 2\}$

### [DÉMONSTRATION] $A \cap B \subseteq A$

Soit  $x \in A \cap B$  quelconque.

Par définition de l'intersection :  $x \in A$  ET  $x \in B$ .

En particulier :  $x \in A$ .

Donc tout élément de  $A \cap B$  est dans  $A$ , ce qui signifie  $A \cap B \subseteq A$ .  $\square$

### [CODE PYTHON]

---

```
# Les ensembles Python = set()
```

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5, 6}
```

```
union          = A | B          # A ∪ B
```

```
intersection   = A & B          # A ∩ B
```

```
difference     = A - B          # A \ B
```

```
sous_ensemble = A <= B          # A ⊆ B ?
```

```
print("A ∪ B :", union)          # {1, 2, 3, 4, 5, 6}
```

```
print("A ∩ B :", intersection)   # {3, 4}
```

```
print("A \ B :", difference)     # {1, 2}
```

```
print("A ⊆ B :", sous_ensemble)  # False
```

```
# Vérifier A ∩ B ⊆ A (notre démonstration)
```

```
print("\nVérification A ∩ B ⊆ A :", intersection <= A)  # True ✓
```

```
# Ensemble vide
```

```
vide = set()
```

```
print("Ensemble vide :", vide, "| Cardinal :", len(vide))
```

```
# Ensemble des parties (power set)
```

```
def power_set(s):
```

```
    """Génère tous les sous-ensembles d'un ensemble"""
```

```
    s = list(s)
```

```
    n = len(s)
```

```
    return [frozenset(s[j] for j in range(n) if (i >> j) & 1)
            for i in range(2**n)]
```

```
parties = power_set({1, 2, 3})
```

```
print("\nEnsemble des parties de {1,2,3} :")
```

```
for p in sorted(parties, key=len):
```

```
    print(" ", set(p) if p else "∅")
```

```
print("Nombre de parties :", len(parties), "= 2³")
```

---

### [THÉORIE]

Une fonction mathématique C'EST une fonction Python :  
elle prend un input (antécédent), elle donne un output (image).  
La différence : en maths on précise exactement les ensembles  
de départ (domaine) et d'arrivée (codomaine).

### [DÉFINITION]

Une fonction  $f : A \rightarrow B$  est une règle qui associe à chaque  
élément  $x$  de  $A$  exactement UN élément  $f(x)$  de  $B$ .

$A$  = domaine (ensemble de départ)  
 $B$  = codomaine (ensemble d'arrivée)  
 $f(x)$  = image de  $x$  par  $f$   
 $\text{Im}(f) = \{f(x) \mid x \in A\}$  = ensemble de toutes les images ( $\subseteq B$ )

Propriétés importantes :

$f$  est INJECTIVE :  $x_1 \neq x_2 \Rightarrow f(x_1) \neq f(x_2)$  (pas de doublons en sortie)  
 $f$  est SURJECTIVE :  $\forall y \in B, \exists x \in A, f(x) = y$  (tout  $B$  est atteint)  
 $f$  est BIJECTIVE : injective ET surjective (correspondance parfaite)

### [EXEMPLE]

$f : \mathbb{R} \rightarrow \mathbb{R}, f(x) = x^2$   
 $f(3) = 9, f(-3) = 9 \rightarrow$  PAS injective (deux  $x$  pour une même image)  
 $f(-1) = 1 < 0$  impossible  $\rightarrow$  PAS surjective sur  $\mathbb{R}$  (les images sont  $\geq 0$ )

$g : \mathbb{R} \rightarrow [0, +\infty), g(x) = x^2$   
Pas injective mais SURJECTIVE sur  $[0, +\infty)$

$h : [0, +\infty) \rightarrow [0, +\infty), h(x) = x^2$   
Injective ET surjective = BIJECTIVE sur ces ensembles

### [DÉMONSTRATION] $f(x) = 2x + 1$ est bijective de $\mathbb{R}$ vers $\mathbb{R}$

INJECTIVITÉ : soient  $x_1, x_2 \in \mathbb{R}$  tels que  $f(x_1) = f(x_2)$   
 $2x_1 + 1 = 2x_2 + 1$   
 $2x_1 = 2x_2$   
 $x_1 = x_2 \quad \checkmark$  donc  $f$  est injective.

SURJECTIVITÉ : soit  $y \in \mathbb{R}$  quelconque. Cherchons  $x$  tel que  $f(x) = y$ .  
 $2x + 1 = y \rightarrow x = (y-1)/2 \in \mathbb{R}$   
 $f((y-1)/2) = 2 \cdot (y-1)/2 + 1 = (y-1) + 1 = y \quad \checkmark$   
Donc  $\forall y \in \mathbb{R}, \exists x \in \mathbb{R}, f(x) = y$  :  $f$  est surjective.

$f$  est bijective.  $\square$   
Sa bijection réciproque :  $f^{-1}(y) = (y-1)/2$

### [CODE PYTHON]

---

# Fonctions : injection, surjection, bijection

```
def est_injective(f, domaine):
    """Teste si f est injective sur un domaine fini"""
    images = [f(x) for x in domaine]
    return len(images) == len(set(images)) # pas de doublons

def est_surjective(f, domaine, codomaine):
    """Teste si f est surjective : toutes les valeurs du codomaine sont atteintes"""
    images = set(f(x) for x in domaine)
    return set(codomaine) <= images

# Exemple 1 : f(x) = 2x + 1 sur un domaine fini
f = lambda x: 2*x + 1
domaine = list(range(-5, 6)) # [-5, ..., 5]
codomaine = [2*x+1 for x in domaine] # images exactes

print("f(x) = 2x+1 :")
print(" Injective :", est_injective(f, domaine)) # True
print(" Surjective:", est_surjective(f, domaine, codomaine)) # True

# Exemple 2 : g(x) = x^2 – pas injective
g = lambda x: x**2
```



```

print("\ng(x) = x2 :")
print(" Injective :", est_injective(g, domaine))      # False

# Composition de fonctions
def compose(f, g):
    """Retourne f ◦ g"""
    return lambda x: f(g(x))

h = compose(lambda x: x + 1, lambda x: x**2)    # h(x) = x2 + 1
print("\nh = f◦g où f(x)=x+1, g(x)=x2")
print("h(3) =", h(3))      # 10 = 32 + 1
print("h(4) =", h(4))      # 17 = 42 + 1

# Fonction inverse
f_inv = lambda y: (y - 1) / 2    # f-1(y) = (y-1)/2
print("\nf(f-1(7)) =", f(f_inv(7)))    # 7.0 ✓
print("f-1(f(3)) =", f_inv(f(3)))    # 3.0 ✓

```

---

## 7. LES RELATIONS – comparer et ordonner

---

### [THÉORIE]

Une relation c'est une façon de "comparer" deux éléments.  
 "est inférieur à", "est divisible par", "est de la même couleur que"...  
 Certaines relations ont des propriétés spéciales qui les rendent utiles.

### [DÉFINITION]

Une relation R sur un ensemble E est un sous-ensemble de  $E \times E$ .  
 On note  $a R b$  pour dire " $(a, b) \in R$ " (" $a$  est en relation avec  $b$ ").

#### Propriétés :

|                |                                        |                                                |
|----------------|----------------------------------------|------------------------------------------------|
| RÉFLEXIVE      | : $\forall a \in E, \quad a R a$       | ("tout élément est en relation avec lui-même") |
| SYMÉTRIQUE     | : $a R b \Rightarrow b R a$            | ("la relation marche dans les deux sens")      |
| ANTISYMÉTRIQUE | : $a R b$ et $b R a \Rightarrow a = b$ | ("si les deux sens, alors égalité")            |
| TRANSITIVE     | : $a R b$ et $b R c \Rightarrow a R c$ | ("on peut "sauter" les intermédiaires")        |

RELATION D'ÉQUIVALENCE = réflexive + symétrique + transitive  
 RELATION D'ORDRE = réflexive + antisymétrique + transitive

### [EXEMPLE]

" $\leq$ " sur  $\mathbb{R}$  :

- Réflexive :  $a \leq a$  ✓ (tout réel est  $\leq$  à lui-même)
- Antisymétrique:  $a \leq b$  et  $b \leq a \Rightarrow a = b$  ✓
- Transitive :  $a \leq b$  et  $b \leq c \Rightarrow a \leq c$  ✓

→ C'est une relation d'ordre.

" $\equiv \pmod{n}$ " sur  $\mathbb{Z}$  (même reste dans la division par  $n$ ) :

- Réflexive :  $a \equiv a \pmod{n}$  ✓
- Symétrique :  $a \equiv b \Rightarrow b \equiv a$  ✓
- Transitive :  $a \equiv b$  et  $b \equiv c \Rightarrow a \equiv c$  ✓

→ C'est une relation d'équivalence.

### [CODE PYTHON]

---

#### # Vérifier les propriétés d'une relation

```

def est_reflexive(R, E):
    return all(R(a, a) for a in E)

def est_symetrique(R, E):
    return all(R(b, a) for a in E for b in E if R(a, b))

def est_antisymetrique(R, E):
    return all(a == b for a in E for b in E if R(a, b) and R(b, a))

def est_transitive(R, E):
    return all(R(a, c) for a in E for b in E for c in E
               if R(a, b) and R(b, c))

E = list(range(1, 6))    # {1, 2, 3, 4, 5}

# Relation ≤

```

```

inferieur = lambda a, b: a <= b
print("Relation ≤ :")
print("  Réflexive      :", est_reflexive(inferieur, E))      # True
print("  Antisymétrique :", est_antisymetrique(inferieur, E)) # True
print("  Transitive     :", est_transitive(inferieur, E))      # True
print("  → ORDRE ✓")

# Relation "même parité"
meme_parite = lambda a, b: a % 2 == b % 2
print("\nRelation 'même parité' :")
print("  Réflexive      :", est_reflexive(meme_parite, E))      # True
print("  Symétrique      :", est_symetrique(meme_parite, E))      # True
print("  Transitive      :", est_transitive(meme_parite, E))      # True
print("  → ÉQUIVALENCE ✓")

# Classes d'équivalence de "même parité"
classes = {}
for a in E:
    cle = a % 2 # représentant de la classe
    classes.setdefault(cle, []).append(a)
print("\nClasses d'équivalence :", list(classes.values()))
# [[2,4], [1,3,5]] ou similaire

```

---

## 8. LES NOMBRES – compter, mesurer, calculer

---

### [THÉORIE]

Les nombres ne sont pas tous "pareils". Chaque famille de nombres a été inventée pour résoudre un problème précis.  
C'est exactement comme les types en Python : int, float, complex...

### [DÉFINITION]

$\mathbb{N} = \{0, 1, 2, \dots\}$  → compter (int  $\geq 0$  en Python)  
 $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$  → dettes (int en Python)  
 $\mathbb{Q} = \{p/q\}$  → partager (fractions.Fraction)  
 $\mathbb{R}$  = droite numérique → mesurer (float en Python, approx.)  
 $\mathbb{C} = a + bi$  → tout résoudre (complex en Python)

Inclusion :  $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$

### [DÉMONSTRATION] $\sqrt{2} \notin \mathbb{Q}$ (déjà prouvée au-dessus, résumé)

Si  $\sqrt{2} = p/q$  irréductible →  $p^2 = 2q^2$  →  $p$  pair →  $p=2k$   
 →  $4k^2 = 2q^2$  →  $q^2 = 2k^2$  →  $q$  pair → contradiction pgcd=1. □

### [CODE PYTHON]

---

```

import math
from fractions import Fraction

# Les différents types de nombres

# N et Z
n = 42      # entier naturel
z = -7      # entier relatif

# Q – fractions exactes (pas d'erreur d'arrondi)
q1 = Fraction(1, 3)
q2 = Fraction(1, 6)
print("1/3 + 1/6 =", q1 + q2) # 1/2 (exact)
print("1/3 + 1/6 =", 1/3 + 1/6) # 0.49999... (approximation float)

# R – approximation flottante
pi_approx = math.pi
print("\nπ ≈", pi_approx)

# C – nombres complexes natifs Python
c1 = 3 + 4j
c2 = 1 - 2j
print("\nc1 =", c1)
print("c2 =", c2)
print("c1 + c2 =", c1 + c2) # (4+2j)
print("|c1| =", abs(c1)) # 5.0 (module = √(3²+4²))

```

```

print("c1.real =", c1.real)      # 3.0
print("c1.imag =", c1.imag)      # 4.0

# Démonstration :  $i^2 = -1$ 
i = 1j
print("\ni² =", i**2)             # (-1+0j) ✓

# Résoudre  $x^2 + 1 = 0$  dans  $\mathbb{C}$ 
import cmath
solutions = [cmath.sqrt(-1), -cmath.sqrt(-1)]
print("Solutions de  $x^2+1=0$  :", solutions) # [1j, -1j]

```

---



---

## 9. DIVISIBILITÉ ET ARITHMÉTIQUE – les briques des entiers

---

### [THÉORIE]

L'arithmétique c'est l'étude des entiers. Les notions de divisibilité, de nombres premiers, de pgcd sont les fondations de la cryptographie, des algorithmes et de toute l'algèbre.

### [DÉFINITION]

$a \mid b$  ("a divise b") :  $\exists k \in \mathbb{Z}, b = k \cdot a$

**DIVISION EUCLIDIENNE** :  $\forall a \in \mathbb{Z}, \forall b \in \mathbb{Z}^*, \exists! (q, r) \in \mathbb{Z} \times \mathbb{N}$  tels que :  
 $a = b \cdot q + r$  avec  $0 \leq r < |b|$   
 $q$  = quotient,  $r$  = reste =  $a \bmod b$

**NOMBRE PREMIER**  $p$  :  $p \geq 2$  et ses seuls diviseurs sont 1 et lui-même.

$\text{PGCD}(a, b)$  = Plus Grand Commun Diviseur (algorithme d'Euclide)  
 $\text{PPCM}(a, b)$  = Plus Petit Commun Multiple =  $|a \cdot b| / \text{pgcd}(a, b)$

**[DÉMONSTRATION]** Algorithme d'Euclide –  $\text{pgcd}(a, b) = \text{pgcd}(b, a \bmod b)$   
On veut montrer que  $\text{pgcd}(a, b) = \text{pgcd}(b, r)$  où  $a = bq + r$ .

Soit  $d = \text{pgcd}(a, b)$ .  
 $d \mid a$  et  $d \mid b \rightarrow d \mid (a - bq) = r \rightarrow d \mid r$   
Donc  $d$  est un diviseur commun de  $b$  et  $r$ .

Soit  $d'$  un diviseur commun quelconque de  $b$  et  $r$ .  
 $d' \mid b$  et  $d' \mid r \rightarrow d' \mid (bq + r) = a$   
Donc  $d' \mid a$  et  $d' \mid b \rightarrow d' \leq \text{pgcd}(a, b) = d$

Donc  $\text{pgcd}(b, r) = d = \text{pgcd}(a, b)$ .  $\square$

### [CODE PYTHON]

---

```

# Arithmétique fondamentale

# Division euclidienne
a, b = 37, 5
q, r = divmod(a, b)
print(f"Division euclidienne : {a} = {b}x{q} + {r}")
# 37 = 5x7 + 2

# Algorithme d'Euclide (récuratif – exactement la démonstration ci-dessus)
def pgcd(a, b):
    """pgcd par algorithme d'Euclide"""
    while b != 0:
        a, b = b, a % b    # exactement : pgcd(a,b) = pgcd(b, a mod b)
    return a

def ppcm(a, b):
    return abs(a * b) // pgcd(a, b)

print("\npgcd(48, 18) =", pgcd(48, 18))    # 6
print("ppcm(4, 6)    =", ppcm(4, 6))       # 12

# Crible d'Ératosthène – trouver tous les nombres premiers ≤ n
def crible_eratosthene(n):
    """Retourne la liste des nombres premiers ≤ n"""
    est_premier = [True] * (n + 1)

```

```

est_premier[0] = est_premier[1] = False
for i in range(2, int(n**0.5) + 1):
    if est_premier[i]:
        for j in range(i*i, n+1, i): # marquer les multiples
            est_premier[j] = False
return [i for i, v in enumerate(est_premier) if v]

premiers = crible_eratosthene(50)
print("\nNombres premiers ≤ 50 :", premiers)

# Décomposition en facteurs premiers
def facteurs_premiers(n):
    facteurs = []
    d = 2
    while d * d <= n:
        while n % d == 0:
            facteurs.append(d)
            n //= d
        d += 1
    if n > 1:
        facteurs.append(n)
    return facteurs

print("\nDécomposition de 360 :", facteurs_premiers(360))
# [2, 2, 2, 3, 3, 5] → 360 = 2³ × 3² × 5

# Théorème de Bézout : pgcd(a,b) = ua + vb
def bezout(a, b):
    """Retourne (u, v) tels que ua + vb = pgcd(a,b)"""
    if b == 0:
        return 1, 0
    u, v = bezout(b, a % b)
    return v, u - (a // b) * v

u, v = bezout(48, 18)
g = pgcd(48, 18)
print(f"\nBézout : {u}×48 + {v}×18 = {u*48 + v*18} = pgcd(48,18) = {g}")

```

---

## 10. RÉSUMÉ – LES RÈGLES D'OR DU RAISONNEMENT MATHÉMATIQUE

---

|                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>RÈGLE 1 – Définir avant d'utiliser</b><br/>         Comme en code : déclarer une variable avant de l'utiliser.<br/>         En maths : définir ce que sont vos objets avant de raisonner.</p>                                                                                                                                                                                                      |
| <p><b>RÈGLE 2 – Un contre-exemple suffit à réfuter un <math>\forall</math></b><br/>         Pour invalider "tous les entiers sont pairs", un seul suffit : 3.</p>                                                                                                                                                                                                                                        |
| <p><b>RÈGLE 3 – Pour prouver <math>\forall</math>, raisonner sur un x QUELCONQUE</b><br/>         Pas sur <math>x=2</math> ou <math>x=5</math> spécifiquement. Sur "un x générique".</p>                                                                                                                                                                                                                 |
| <p><b>RÈGLE 4 – L'implication <math>P \Rightarrow Q \neq</math> causalité</b><br/>         Si P est faux, <math>P \Rightarrow Q</math> est toujours vrai peu importe Q.</p>                                                                                                                                                                                                                              |
| <p><b>RÈGLE 5 – Choisir la bonne stratégie de preuve</b><br/>         Direct → quand la chaîne logique est claire<br/>         Contraposée → quand <math>\neg Q \Rightarrow \neg P</math> est plus simple que <math>P \Rightarrow Q</math><br/>         Absurde → quand supposer <math>\neg P</math> mène clairement à une impasse<br/>         Récurrence → quand la propriété dépend d'un entier n</p> |
| <p><b>RÈGLE 6 – Vérifier <math>\neq</math> Démontrer</b><br/>         Vérifier sur 1000 exemples ne PROUVE pas le cas général.<br/>         La preuve doit couvrir TOUS les cas, même infinis.</p>                                                                                                                                                                                                       |

[CODE PYTHON FINAL – Mini-framework de vérification logique]

# Un outil pour explorer la logique interactivement

```

def analyser_implication(P_func, Q_func, domaine, description=""):
    """

```

```

Analyse  $P \Rightarrow Q$  sur un domaine fini.
Trouve les cas où P est vrai, où Q est vrai, et détecte
d'éventuels contre-exemples.
"""
print(f"\nAnalyse de : {description}")
contre_exemples = []
cas_P_vrai = []

for x in domaine:
    p = P_func(x)
    q = Q_func(x)
    if p:
        cas_P_vrai.append(x)
    if p and not q:
        contre_exemples.append(x)

print(f"  Domaine : {len(domaine)} valeurs")
print(f"  P vraie pour : {cas_P_vrai[:10]}{'...' if len(cas_P_vrai)>10 else ''}")
if contre_exemples:
    print(f"    □ CONTRE-EXEMPLES trouvés : {contre_exemples[:5]}")
    print(f"    → L'implication  $P \Rightarrow Q$  est FAUSSE")
else:
    print(f"    ✓ Aucun contre-exemple. L'implication SEMBLE vraie sur ce domaine.")
    print(f"    (Attention : ceci ne PROUVE pas le cas général !)")

domaine = list(range(-20, 21))

# Test 1 :  $x > 0 \Rightarrow x^2 > 0$  (vraie pour  $x \neq 0$ )
analyser_implication(
    lambda x: x > 0,
    lambda x: x**2 > 0,
    domaine,
    "x > 0  $\Rightarrow$  x² > 0"
)

# Test 2 :  $x^2 > 0 \Rightarrow x > 0$  (fausse : contre-exemple x=-1)
analyser_implication(
    lambda x: x**2 > 0,
    lambda x: x > 0,
    domaine,
    "x² > 0  $\Rightarrow$  x > 0"
)

```

---

```

#####
#
# CE QUE TU MAÎTRISES MAINTENANT :
#
# ✓ Proposition      = le booléen des maths (True/False)
# ✓ Connecteurs     = and, or, not, => , <=>
# ✓ Quantificateurs = for all (∀) et exists (∃) = all() et any()
# ✓ 4 types de preuve : directe, contraposée, absurde, récurrence
# ✓ Ensembles       = set Python, avec union/intersection/différence
# ✓ Fonctions       = fonctions Python + notion injection/surjection/bijection
# ✓ Relations       = réflexive, symétrique, transitive → ordre / équivalence
# ✓ Nombres         =  $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$ 
# ✓ Divisibilité    = pgcd, ppcm, nombres premiers, Euclide, Bézout
#
# PROCHAINE ÉTAPE NATURELLE :
#   → Algèbre (équations, polynômes, systèmes)
#   → Analyse (limites, dérivées, intégrales)
#   → Probabilités (espaces, variables aléatoires)
#
#####

```